

Inigo Quilez :: articles :: making textures with gm.dls - 2008

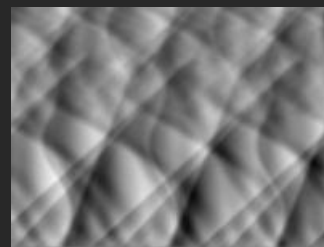


Intro

It's a well known fact that cheating is an important factor in 4 kilobytes (and only 4 kb) intros. The best known example is the now widely used and accepted gm.dls trick - in case you still don't know it, it's all about a 3 megabyte file that contains almost five hundred wav files, ready to be triggered by your module player or to be input to any machine of a soft synthesizer. Even if these wavs sound crappy, it solves a big part of the sound generation problem at least. However for textures, the problem still remains, as can be seen in the fact that 90% of the 4k intros lack textures. I think if we had an equivalent to gm.dls for textures, it would become widely used too (again, as source of textures, or just as source of patterns in a complex texture generation system). Well, we have all those wallpapers of Windows; the problem is they are different between OS versions, unlike gm.dls. So let's think of something else in a loud voice... "This is not easy... but shit, but we *need* to find something, think... let's see, we need a file that is as standard as gm.dls that contains textures... hm... a file as standard as gm.dls... standard as gm.dls... as gm.dls. Got it! The answer is gm.dls!"

gm.dls, source of textures:

So we are going to try to find textures in a collection of wav files? It's not obvious this is going to work at all. Why should anything in gm.dls resemble a texture, and not noise? Well, actually getting noise from a digital file would be quite surprising, yet useful. Back to gm.dls, when displaying a wav file as an image what we would expect to see is some kind of unstructured random patterns (not sure "random patterns" makes sense, but well, you get it). Luckily sounds are not random but quite smooth. But sounds are 1D signals, and what we look for is 2D signals. So, it's clear what we have to do, take the 1D signal, split it in equally-sized chunks, and put each chunk below the others to form a stack of 1D signals - an image. If you ever played with sound or any other kind of 1D signals, you'll probably see by now that this process will not create any nice looking image unless these chunk sizes are chosen in a "magical" way such that each chunk is



wav 5, pitch 440 Hz

similar to the previous ones, so that each row of the image makes sense when put together with the other rows.

Luckily again for us, sound is all about oscillations of periodic patterns. Indeed, real sounds are not periodic (that would be terrible boring to listen to), but they do contain quite an amount of periodic power (watts). Basically, what I say is that the Fourier transform (well, the spectral density of a signal) normally has some nice peaks at given frequencies that make actually a sound sound like that. Without those peaks, without any dominant periodicities over others (flat spectrum), the thing would sound like noise. What all this means is that the signals (wavs) in gm.dls will certainly contain self-similarities or repetitions, because they are sounds! Meaning that it's not a miracle or anything magical that for each wav file in gm.dls, we can find a set of values, so that when the wav is split in chunks of that size we get a collection of similar chunks. And this leads to a coherent 2D image...



wav 388, pitch 259 Hz

OK, let's just try and see

You know what to do now. Open gl.dms, search for "data" sequence, read 4 more bytes with the length of the sample, read as many samples as you just indicated, store the sound buffer, and go for the next "data" chunk. For each sound buffer, split it in chunks and construct the image. As indicated before, the chunk size should match the main frequency peak of the signal. There are also other values that make interesting patterns. For example: all multiples of that value (harmonics). Also look to the second and third main frequency peaks, if they are not already harmonics of the main one, they will actually create very different yet interesting patterns. But don't worry, you don't need to program any DFT or self-correlation here. Just try different values in brute force, you will immediately see which values are the good ones, because as you approach a good pitch value the image changes from an ugly thing into an interesting shape. Harmonic values will normally lead you to tiled/stretched versions of the main image. Depending on how inharmonic the sound is, you can get more than one very different patterns (inharmonic signals contain several hidden images, as opposed to harmonic ones that only contain one!).

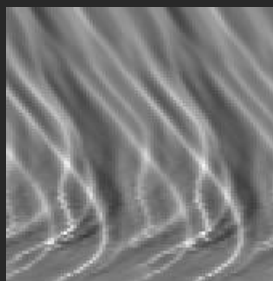
When I came up with the idea, I could not resist to try it. In a few minutes I got 70 useful textures. Not bad for

a sound repository! I added a few of them to this article. Note that these are NOT photoshopped at all, what you see is the raw data coming from the gm.dls file, believe it or not. Some of them look like they had a lighting effect (emboss) - that comes also from the gm.dls file and it's probably due to the fact that the underlying sound was high-pass or band-stop filtered.

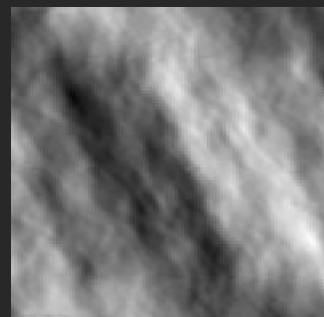
Another side effect of dealing with periodic signals is that most textures tile quite well, at least horizontally. They are not fantastic textures as you can see below, but remember they come for free!



wav 117, pitch 296 Hz



wav 54, pitch 519 Hz



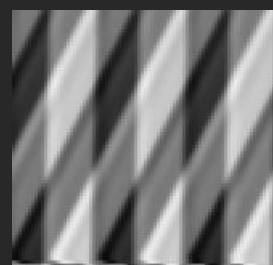
wav 405, pitch 440 Hz



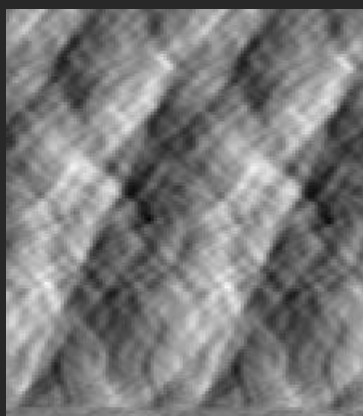
wav 121, pitch 496 Hz



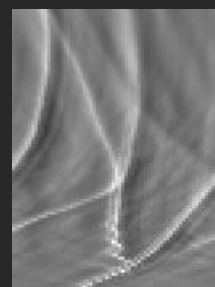
wav 377, pitch 261 Hz



wav 386, pitch 525 Hz



wav 391, pitch 394 Hz



wav 53, pitch 668 Hz